Creación de Rate Limiter Web a través de IA generativa

Integrantes:

Conforti, Luis Alberto
Delgado, Matias Agustin
Menchaca, Brian Angel Gaston
Porcile, Franco
Rodriguez, Mariano Lionel

lconforti294@alumno.unlam.edu.ar
matdelgado@alumno.unlam.edu.ar
bmenchaca@alumno.unlam.edu.ar
fporcile995@alumno.unlam.edu.ar
marianolrodriguez@alumno.unlam.edu.ar

Asignatura:

3658 - Programación Concurrente

Fecha:

21/10/2026

I. Introducción

El presente informe pretende dar a cuenta sobre el resultado de la experimentación con herramientas de vibe coding en el desarrollo de proyectos de software (particularmente aquellos centrados en la resolución de problemas concurrentes), y la medición de su rendimiento mediante el uso de métricas y rúbricas.

II. Desarrollo

A. Definición de la problemática

Para la experimentación solicitada, hemos ideado una problemática concurrente sencilla pero de complejidad no trivial. La misma se define de la siguiente manera:

“Dado un servidor web que no posee ningún mecanismo limitación de tasa de peticiones por cliente, se propone incorporar un rate limiter que funcione, almacenando por cada cliente (con un id, una dirección IP, una fingerprint, o cualquier clase de identificación unívoca) un historial con la cantidad de peticiones realizadas dentro de una ventana temporal. Por ejemplo: Cada cliente podría realizar como máximo 5 (cinco) peticiones en un intervalo de 10 (diez) segundos, rechazando una sexta si se produce dentro de ese intervalo.

En una situación donde un cliente ya realizó 4 peticiones dentro de ese intervalo, considerando que el servidor procesa cada nueva petición con un hilo, se produce un problema: De realizar el cliente dos peticiones nuevas en el mismo instante, se crearían dos hilos nuevos y ambos podrían llegar a leer el historial de peticiones del cliente actual en la ventana de tiempo. Al no haberse completado las 5 (límite), se permitiría el procesamiento de ambas, admitiendo 6 peticiones dentro de la ventana temporal, excediendo el límite. Esto genera una condición de carrera donde dependiendo del orden de ejecución de los hilos creados, puede excederse o no el límite.

Si bien podría resolverse limitando el acceso al historial de peticiones (que podría ser un hashmap [2]) a un hilo a la vez, se limitaría la performance, dado que se pasaría de un modelo concurrente a uno secuencial. Debe resolverse este problema de forma concurrente y sin condiciones de carrera.”.

B. Refinamiento del prompt

Definida la problemática que la herramienta de IA debía resolver para nuestra investigación, nuestro próximo paso fue el refinamiento de la misma. Si bien era posible la generación de código directa mediante el uso de esta problemática como un prompt simple en lenguaje natural, hemos decidido refinarla, llevándola a un prompt con enfoque más técnico; conteniendo una estructura más detallada, una serie de requisitos a cumplir, ejemplos, casos de prueba obligatorios y benchmarks para obtener un desarrollo más completo desde un inicio.

Este refinamiento del prompt inicial fue realizado mediante el uso de Claude Sonnet 5.5, utilizando únicamente la solicitud de generación de un prompt técnico estructurado con los elementos mencionados previamente y la definición de la problemática. El resultado fue un archivo en formato markdown que consta de 13 secciones:

1. *Resumen del problema.*
2. *El desafío central.*
3. *Requisitos funcionales.*
4. *Requisitos no funcionales.*
5. *Diseño recomendado.*
6. *Puntos delicados de la implementación.*
7. *Arquitectura propuesta del proyecto (en directorios).*
8. *Estrategia de tests.*
9. *Benchmark.*
10. *Criterios de aceptación.*
11. *Objetivos fuera de alcance.*
12. *Stack tecnológico.*
13. *Instrucciones específicas para la IA que lo implemente.*

Si bien se adjunta junto a este informe dicho archivo que contiene el prompt técnico detallado con estas secciones, es un detalle no menor a mencionar que la herramienta propuso, además de las secciones con las pautas especificadas, una sección con un diseño mínimo recomendado y una estructura base. Hemos decidido mantener estas secciones en el documento ya que consideramos que le servirían como una guía de mayor precisión a la herramienta de IA que finalmente implementaría este prompt técnico.

Otro apartado que consideramos importante a mencionar es el de la adición de un requisito funcional que no contemplamos originalmente pero Claude incluyó: La limpieza de memoria, eliminando entradas antiguas para liberar los recursos de memoria cuando estos no se utilizan. Lo consideramos un agregado de utilidad por lo que decidimos mantenerlo.

C. Generación de código

Utilizando el prompt técnico creado previamente, se realizó la pertinente generación de código fuente. Para esto, se utilizó Codex 5.6 Sol, integrado en un entorno de desarrollo integrado (IDE, por sus siglas en inglés), por lo que no fue necesario transcribir ningún fragmento de código, como podría ocurrir con herramientas sin esta integración.

La implementación fue generada en Java 21, utilizando Maven 3.8 para el uso de ciertas dependencias para un sencillo establecimiento de un servidor HTTP básico sobre el cual ejecuta el rate limiter, como también para el uso de las bibliotecas de testing JUnit y benchmarking JMH [3], favoreciendo la legibilidad y verificación de calidad del código.

Esta presenta tres alternativas para la resolución del problema concurrente: El limitador por clave, el lock global y la forma “ingenua”, generadora de condiciones de carrera. En los tres casos se utiliza un hashmap para identificar a cada cliente y su “bucket” o lista de peticiones

que poseen las marcas temporales del momento de recepción de cada una, permitiendo obtener la cantidad de peticiones realizadas en el intervalo definido del limitador. A continuación se explica brevemente la estrategia utilizada por cada alternativa:

1. Limitador por clave: El historial de peticiones fue creado con un hashmap concurrente, una estructura que habilita a los distintos hilos que la acceden y modifican trabajar de forma atómica, de modo que los hilos que procesan peticiones de un mismo cliente ejecuten de forma secuencial, pero no restrinjan la operación concurrente de otros hilos atendiendo las peticiones de otros clientes.

2. Lock global: El historial fue creado con un hashmap no atómico: Su acceso concurrente puede generar la condición de carrera mencionada al inicio del informe. Para solventar esto, se utiliza un semáforo mutex global que convierte el acceso concurrente de los hilos a un esquema secuencial, sin importar si las peticiones que atienden son sobre el mismo cliente o distintos, limitando la performance pero sirviendo como línea de base para las métricas.

3. Con condición de carrera: Replica la implementación del limitador por clave, pero en lugar de manejar la lista de peticiones directamente sobre el hashmap concurrente de forma atómica, genera una copia local, la modifica y luego escribe esa copia en el hashmap, generando una condición de carrera, sin resolver el problema original. Sirve como demostración de que incluso utilizando una estructura apta para el manejo concurrente, puede no resolverse el problema si no se utiliza adecuadamente.

D. Análisis en la implementación de código

Analizando la funcionalidad principal del proyecto, entre las alternativas de limitadores expuestas previamente, se puede comprobar que aquella que la herramienta de IA ha utilizado como la principal es el limitador por clave. Este mismo, efectivamente, resuelve la problemática planteada, gracias al uso del hashmap concurrente y sus métodos de ejecución atómica. Estos garantizan la mutua exclusión en la región crítica entre hilos que trabajen con peticiones de un mismo cliente, serializando su trabajo sin impedir que otros hilos para peticiones de otros clientes trabajen concurrentemente sobre el historial, simultáneamente, evitando las condiciones de carrera con la mejor performance posible (esto es constatado con las métricas más adelante en el apartado de resultados).

Dado que esta implementación no utiliza semáforos de forma directa (si no en el manejo interno del hashmap concurrente) y su único recurso de uso exclusivo es el historial, no existe posibilidad de generar una espera circular, eliminando la posibilidad de ocurrencia de deadlocks.

A su vez, la limpieza de peticiones de clientes inactivos se realiza correctamente con accesos atómicos al historial, sin permitir que la limpieza de peticiones antiguas de un cliente ingrese en su sección crítica en el mismo momento en el que lo realiza la recepción de peticiones nuevas, por lo que se garantiza la mutua exclusión en este agregado. No es

menor mencionar que la clase que realiza la limpieza fue creada únicamente para este tipo de limitador y no para los demás.

En términos de sincronización, si bien en la implementación principal y secundarias la herramienta de IA solo utilizó semáforos mutex (para la implementación de lock global) y ejecución métodos atómicos del hashmap concurrente, podemos ver reflejado el uso de barreras para hilos en el apartado de tests. Estas tienen por objetivo iniciar el trabajo de todos los hilos a la vez generando la mayor carga de trabajo posible para demostrar que efectivamente el código se encuentra libre de condiciones de carrera.

En términos de calidad en el código fuente, podemos destacar la modularidad que ofrece el mismo; elementos como los limitadores o los mecanismos de identificación de clientes son intercambiables mediante el uso de interfaces. En términos de legibilidad; si bien la resolución de métodos que implementa no requieren una extensa cantidad de líneas de código, el uso de magic numbers (principalmente en tests), la falta de comentarios más allá de los más significativos, y el uso expresiones de inferencia de tipos en la asignación de variables no son buenas prácticas a seguir, las cuales podrían ser corregidas en siguientes iteraciones.

III. Resultados

A. Métricas “de consumo”

Para la obtención de métricas de consumo y su interpretación, hemos utilizado dos procedimientos:

1. *Medición directa de consumo de CPU y memoria utilizada.*
2. *Medición de operaciones por segundo en diferentes situaciones, con diferentes implementaciones del limitador.*

Para el primer tipo, se ha generado un script en Python utilizando la biblioteca *psutil [1]*, de modo que se mide tanto el consumo de CPU y de memoria RAM en dos situaciones particulares: Un solo hilo que genera constantes peticiones al servidor, o 32 hilos generando peticiones de forma concurrente. Ambos casos se ejecutan durante 30 segundos.

Para el segundo tipo se ha ejecutado una prueba de benchmark utilizando la biblioteca JMH. En esta se comparan las dos alternativas que resuelven el problema de la condición de carrera: el lock global y el limitador por clave. Se prueban ambos limitadores con dos casos particulares: peticiones de distintos clientes (claves distintas) o del mismo (misma clave).

Tabla 1: comparativa de mediciones

Número de Hilos	Global, claves distintas	Por clave, claves distintas	Global, misma clave	Por clave, misma clave
1	17,7 M	12,4 M	17,6 M	12,5 M
2	4,8 M	12,9 M	6,7 M	6,0 M
4	6,7 M	16,6 M	7,6 M	3,6 M
8	5,0 M	20,9 M	7,9 M	2,6 M
16	6,4 M	25,1 M	7,4 M	3,0 M

Los valores presentados deben ser tomados únicamente en términos porcentuales de mejora según como cada implementación maneja cada caso.

Expuesto el análisis de implementación, puede observarse como el caso de mejor performance es el uso del limitador por clave cuando se realizan peticiones concurrentes de distintos clientes, y, en contraposición, el peor caso es precisamente se da al realizar todas las peticiones desde un mismo cliente, haciendo la carga de trabajo completamente secuencial, con una performance inclusive peor que el lock global.

B. Métricas “de funcionalidad”

Como se expuso en apartados anteriores, el código generado por la herramienta de IA pudo resolver el problema planteado (en base al prompt técnico indicado) efectivamente, con el único punto negativo a destacar, la falta de implementación de una limpieza de claves para las implementaciones secundarias. Por otro lado, la compilación y ejecución se da sin problemas utilizando las versiones especificadas de Maven y el JDK.

C. Métricas de concurrencia

Como se detalló en el análisis de código, la solución generada por la herramienta IA elimina la posibilidad de condiciones de carrera (incluso mostrando un ejemplo donde podría ocurrir si se implementara de forma errónea esta solución), garantizando la mutua exclusión entre los hilos que en su sección crítica trabajan sobre el historial compartido, impidiendo que el límite de peticiones se exceda, pero a su vez permitiendo la mayor performance posible al trabajar con hilos workers de peticiones para distintos clientes.

IV. Conclusiones

A partir de los resultados obtenidos, podemos concluir que las herramientas de IA generativa son capaces de producir una solución funcional para una problemática concurrente, incluso a partir de un prompt técnico detallado.

Es importante considerar que aunque la problemática expuesta fuese sencilla y su código fuente fuera generado en base a un prompt técnico detallado, la herramienta de IA que lo implementó cometió errores de programación que, si bien pueden considerarse mínimos en el contexto del proyecto, debemos pensar en cuanto escalaría el número e intensidad de los errores en cuanto más complejo y grande sea el proyecto objetivo sin la correcta supervisión de un profesional que sepa guiarla en el caso de utilizar una estrategia de vibe coding, ya sea parcial o total. No se trata solamente de comprobar si la IA genera código que funciona, el cuál es un error muy común enfocarse en sus requisitos funcionales fácilmente observables, sino de evaluar cómo se comporta ese código en términos de funcionalidad, concurrencia, consumo y rendimiento como bien pudimos observar en las rúbricas.

En conclusión, las herramientas de IA generativa constituyen un recurso valioso para el desarrollo de software, pero su utilización debe estar acompañada del criterio técnico del profesional, quien debe evaluar la corrección, eficiencia y calidad de las soluciones generadas. La IA debe funcionar como un potenciador de sus capacidades, no como un reemplazo de su responsabilidad en el proceso de desarrollo.

V. Referencias

[1] psutil. *API Reference* [en línea]. Fecha de creación: no especificada. Fecha de consulta: 19 de octubre de 2026. Disponible en: https://psutil.readthedocs.io/

[2] Oracle. *ConcurrentHashMap (Java SE 21 & JDK 21)* [en línea]. Fecha de creación: no especificada. Fecha de consulta: 19 de octubre de 2026. Disponible en: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html

[3] OpenJDK. *Java Microbenchmark Harness (JMH)* [en línea]. Fecha de creación: no especificada. Fecha de consulta: 19 de octubre de 2026. Disponible en: https://github.com/openjdk/jmh/